

JS Assignment 3

Q1. Create a hierarchy of person, employee and developers.

```
function Person() {
  this.isHuman = true;
}

function Employee() {
  this.isEmployee = true;
}

Employee.prototype = new Person();

function Developers() {
  this.isDeveloper = true;
}

Developers.prototype = new Employee();

var developer = new Developers();
console.log(developer.isHuman);
console.log(developer.isEmployee);
console.log(developer.isDeveloper);
```

Q2 Given an array, say [1,2,3,4,5]. Print each element of an array after 3 secs.

```
382
383   const nums = [1, 2, 3, 4, 5];
384   let index = 0;
385   function printNext() {
386     console.log(nums[index]);
387     index++;
388     let id;
389     if (index === nums.length) {
390       clearTimeout(id)
391     }
392     else {
393       id = setTimeout(printNext, 3000);
394     }
395   }
396   printNext();
397
```

Problems Output Debug Console Terminal Ports Filter C

4
5

[Done] exited with code=0 in 12.224 seconds

[Running] node "/home/aman-mittal/Desktop/htmlAssign/learn.js"

1
2
3
4
5

[Done] exited with code=0 in 12.217 seconds

Q3 Explain the difference between Bind and Call (example).

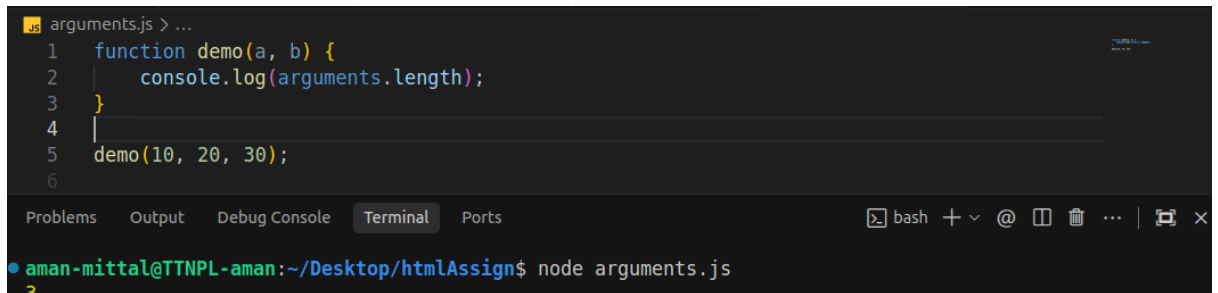
Basis	Function.call()	Function.bind()
Purpose	Immediately invokes the function	Returns a new function with fixed this
Invocation	Executes right away	Needs to be called later
Return value	Returns the function's return value	Returns a new bound function
this handling	this is set only for that call	this is permanently fixed
Arguments	Passed one by one after context	Can be pre-set (partial application)
Syntax	<code>fn.call(context, arg1, arg2)</code>	<code>fn.bind(context, arg1, arg2)</code>
Reusability	Must pass context every time	Context reused automatically
Effect on original function	Does not change original function	Does not change original function
Common use case	Borrowing methods, immediate execution	Event handlers, callbacks
Example	<code>sayHello.call(aman, "Hi")</code>	<code>const f = sayHello.bind(aman)</code>

Q4 Explain 3 properties of argument object.

Ans

(1) Arguments.length :

Returns the number of arguments passed to the function.

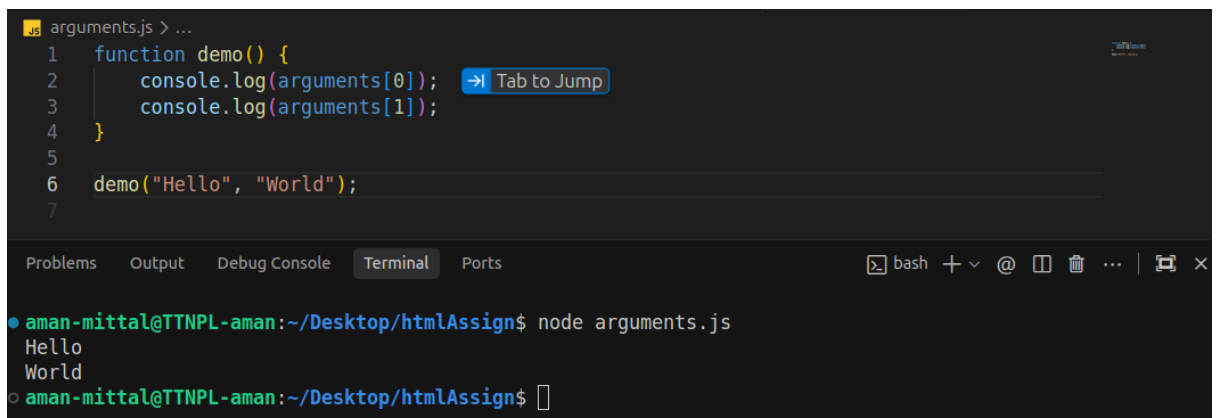


```
arguments.js > ...
1 function demo(a, b) {
2   console.log(arguments.length);
3 }
4
5 demo(10, 20, 30);
6

Problems Output Debug Console Terminal Ports
aman-mittal@TTNPL-aman:~/Desktop/htmlAssign$ node arguments.js
```

(2) arguments[index] :

Access arguments using index notation (like an array).

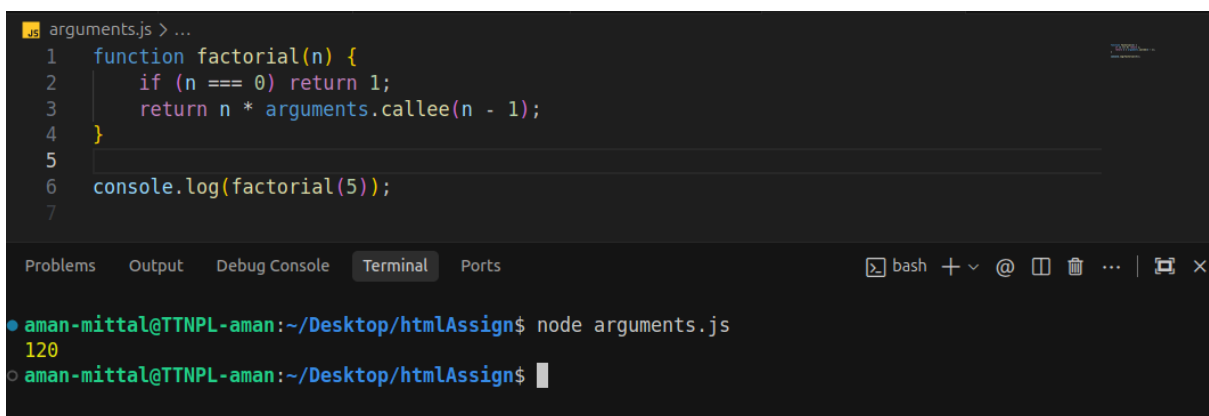


```
arguments.js > ...
1 function demo() {
2   console.log(arguments[0]);
3   console.log(arguments[1]);
4 }
5
6 demo("Hello", "World");
7

Problems Output Debug Console Terminal Ports
aman-mittal@TTNPL-aman:~/Desktop/htmlAssign$ node arguments.js
Hello
World
aman-mittal@TTNPL-aman:~/Desktop/htmlAssign$
```

(3) Arguments.callee :

- Refers to the currently executing function.
- Used earlier for recursive anonymous functions.



```
arguments.js > ...
1 function factorial(n) {
2   if (n === 0) return 1;
3   return n * arguments.callee(n - 1);
4 }
5
6 console.log(factorial(5));
7

Problems Output Debug Console Terminal Ports
aman-mittal@TTNPL-aman:~/Desktop/htmlAssign$ node arguments.js
120
aman-mittal@TTNPL-aman:~/Desktop/htmlAssign$
```

Q5 Create a function which returns number of invocations and number of instances of a function.

HTML File -

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Instances & Invocations</title>
</head>

<body>

  <h1>Instances and Invocations</h1>

  <label for="name">Name:</label>
  <input type="text" id="name">

  <label for="age">Age:</label>
  <input type="number" id="age">
  <button type="button" onclick="createInstance()">Create
Instance</button>
  <button type="button" onclick="createInvocation()">Create
Invocation</button>

  <script src="instances.js"></script>
</body>

</html>
```

JS Code -

```
let instances = 0;
let invocations = 0;

function Person(name, age) {
  invocations++;
  this.name = name;
  this.age = age;
```

```

}

function createInstance() {
  const nameInput = document.getElementById("name");
  const ageInput = document.getElementById("age");

  const person = new Person(nameInput.value, ageInput.value);
  instances++;

  console.log(`Invocation count : ${invocations}, Instance count : 
${instances}`);

  nameInput.value = "";
  ageInput.value = "";
}

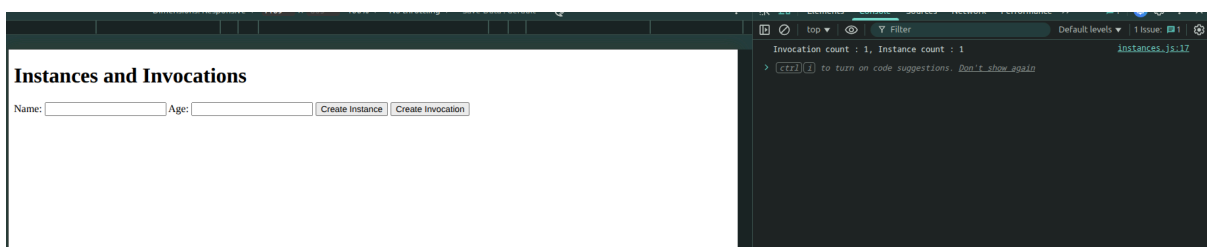
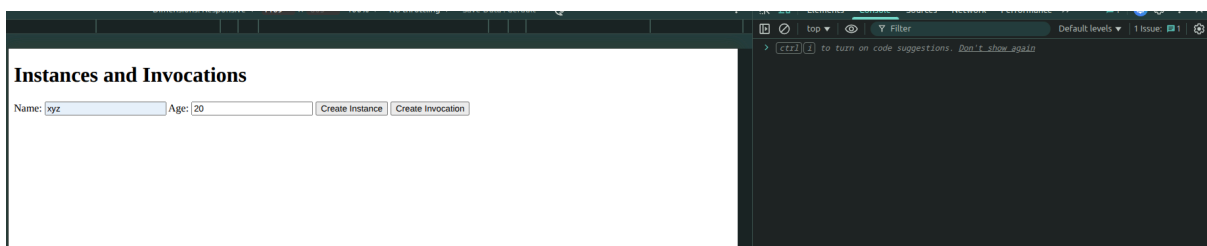
function createInvocation() {
  const nameInput = document.getElementById("name");
  const ageInput = document.getElementById("age");

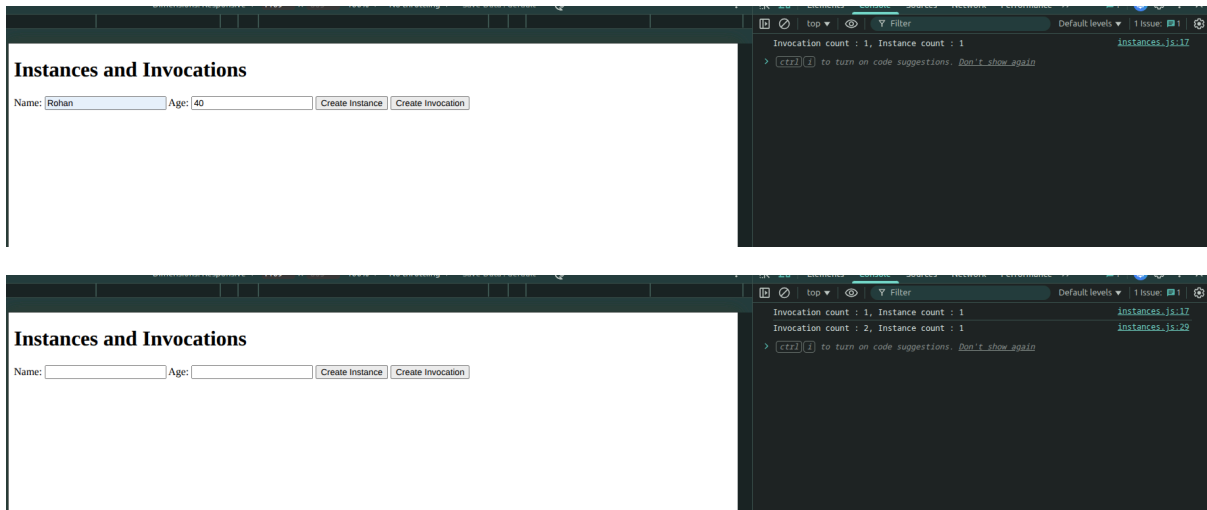
  const person = Person(nameInput.value, ageInput.value);

  console.log(`Invocation count : ${invocations}, Instance count : 
${instances}`);

  nameInput.value = "";
  ageInput.value = "";
}

```





Q6 Create a counter using closures.

```
function counter() {  
  let count = 0;  
  
  return {  
    increment() {  
      count++;  
      return count;  
    },  
    decrement() {  
      count--;  
      return count;  
    },  
    getCount() {  
      return count;  
    }  
  };  
}  
  
const c = counter();  
  
console.log(c.increment());  
console.log(c.increment());  
console.log(c.decrement());  
console.log(c.getCount());
```

Q7 Explain 5 array methods with examples.

Assume:

```
const nums = [1, 2, 3, 4, 5];
```

1. `forEach()`

What it do

- Runs a function for each element in the array
- Mainly used for **side effects** like logging, updating UI, etc.

Return value

- undefined

Original array

- Not modified (unless you manually change elements)

Length

- No result array

Code

```
nums.forEach(num => {  
  console.log(num * 2);  
});
```

Output

4
6
8
10

2. map()

What it does

- Transforms each element in the array
- Always creates a **new array**

Return value

- New array

Original array

- Not modified

Length

- Same as the original array

Code

```
const doubled = nums.map(num => num * 2);  
console.log(doubled);
```

Output

[2, 4, 6, 8, 10]

3. filter()

What it does

- Keeps elements that satisfy a condition

Return value

- New array

Original array

- Not modified

Length

- Less than or equal to original length

Code

```
const even = nums.filter(num => num % 2 === 0);  
console.log(even);
```

Output

```
[2, 4]
```

Use when

- You want to remove elements based on a condition

4. reduce()

What it does

- Combines all elements into a **single value**
- Very powerful and flexible

Return value

- Any type (number, object, array, string, etc.)

Original array

- Not modified

Length

- Single value (not an array)

Code

```
const sum = nums.reduce((acc, curr) => {  
  return acc + curr;  
}, 0);
```

```
console.log(sum);
```

Output

Note

- Can replace `map()` and `filter()` in advanced cases

5. `some()`

What it does

- Checks if **at least one element** passes the condition

Return value

- Boolean (true / false)

Stops early

- Yes (better performance)

Code

```
const hasEven = nums.some(num => num % 2 === 0);  
console.log(hasEven);
```

Output

true

6. `every()`

What it does

- Checks if **all elements** pass the condition

Return value

- Boolean

Stops early

- Yes

Code

```
const allPositive = nums.every(num => num > 0);  
console.log(allPositive);
```

Output

true

7. slice()

What it does

- Extracts a portion of an array

Return value

- New array

Original array

- Not modified

Syntax

`array.slice(start, end);` // end index not included

Code

```
const part = nums.slice(1, 4);  
console.log(part);  
console.log(nums);
```

Output

[2, 3, 4]

[1, 2, 3, 4, 5]

8. splice() (Important)

What it does

- Adds or removes elements

Return value

- Array of removed elements

Original array

- Modified (mutates array)

Syntax

`array.splice(start, deleteCount, item1, item2);`

Code

```
const removed = nums.splice(1, 2, 99, 100);  
console.log(removed);  
console.log(nums);
```

Output

```
[2, 3]
```

```
[1, 99, 100, 4, 5]
```